

PKCERT AI & Software Development Internship

Task 25 – Sequence Modeling: Introduction to Sequence Data, RNN/LSTM Fundamentals & Text Classification

Objective:

Build a rigorous theoretical and practical foundation in sequence modeling. Interns will formalize the properties of sequential data, derive the recurrence relations underlying vanilla RNNs and LSTMs from first principles, implement a forward/backward pass for a recurrent cell manually, and apply these concepts to train a working text classification model using either an LSTM architecture or a pretrained embedding representation.

Total Marks: 100

Part A – Sequence Data & RNN Fundamentals (20 Marks)

- Formally define what distinguishes sequential data (text, time series, audio) from i.i.d. tabular data, and explain why feedforward networks are structurally unsuited to modeling it.
- Derive the recurrence relation for a vanilla RNN cell (hidden state update and output equations) in matrix form, explicitly stating all weight matrices, biases, and dimensions.
- Unroll a vanilla RNN across time steps in a diagram and use it to explain, mathematically, the origin of the vanishing/exploding gradient problem via the repeated multiplication of Jacobians during backpropagation through time (BPTT).
- Derive the BPTT gradient expression for the hidden-to-hidden weight matrix and show, using the derivation, why long-range dependencies are difficult for a vanilla RNN to learn.

Part B – LSTM Theory & Manual Cell Implementation (20 Marks)

- Derive the full set of LSTM equations (forget gate, input gate, candidate cell state, cell state update, output gate, hidden state) and explain the specific mathematical mechanism by which the cell-state highway mitigates vanishing gradients relative to the vanilla RNN derived in Part A.
- Implement a single LSTM cell's forward pass from scratch using only NumPy (no high-level deep learning framework for this step), verified against a known-correct reference implementation (e.g., PyTorch's `nn.LSTMCell`) on identical inputs and weights.
- Implement the backward pass (gradients with respect to all gate weights, biases, and the previous cell/hidden states) for a single time step by hand-deriving and coding the chain rule through the gates; verify correctness using numerical gradient checking.
- Compare and justify, with reasoning grounded in the derivations above, the difference between LSTM and GRU in terms of gating complexity and parameter count.

Part C – Text Classification Mini-Project (45 Marks)

- Select a public text classification dataset (e.g., a sentiment or topic-classification corpus) not used in any previous task, and justify the choice of dataset given the task's sequence-modeling focus.
- Build a complete text preprocessing pipeline: tokenization, vocabulary construction, sequence padding/truncation, and either (a) trainable embeddings initialized from scratch, or (b) a pretrained

embedding representation (e.g., GloVe or a transformer-derived embedding) — justify whichever choice is made.

- Implement and train a text classification model using an LSTM (or bidirectional LSTM) architecture in a deep learning framework of choice, tracking and plotting training/validation loss and accuracy curves across epochs.
- Evaluate the trained model using accuracy, precision, recall, F1-score, and a confusion matrix on a held-out test set.
- Run a controlled ablation comparing at least two configurations (e.g., trainable vs. pretrained embeddings, unidirectional vs. bidirectional LSTM, or varying hidden-state dimensionality) and report the quantitative effect on validation performance and convergence behavior.
- Diagnose and discuss any evidence of overfitting or underfitting observed during training, and describe the mitigation strategy applied (e.g., dropout, early stopping, regularization) along with its measured effect.

Part D – Analysis & Documentation (15 Marks)

- Save the final trained model's weights/parameters using an appropriate serialization method for the framework used.
- Summarize the full pipeline: preprocessing decisions, architecture chosen, embedding strategy, and the reasoning behind each.
- Summarize the quantitative findings from the Part C ablation experiment.
- Discuss at least two non-trivial challenges encountered while deriving the LSTM backward pass or training the classification model, and how each was resolved.
- Reflect critically on the limitations of LSTM-based sequence models relative to attention-based architectures for text classification.

Assessment Breakdown

Assessment Area	Marks
Sequence Data & RNN Fundamentals	20
LSTM Theory & Manual Cell Implementation	20
Text Classification (Mini-Project)	45
Analysis & Documentation	15
Total	100

Note: For Parts A and B's core derivations and the manual LSTM cell implementation, use of high-level automatic-differentiation frameworks (TensorFlow, PyTorch, Keras) is not permitted for the from-scratch computation — the intent is to demonstrate a working understanding of the underlying mathematics. Frameworks may be used for the Part C classification model and for verifying the manual implementation in Part B.